Neural Networks: From Perceptrons to Deep Learning

Lecture 29

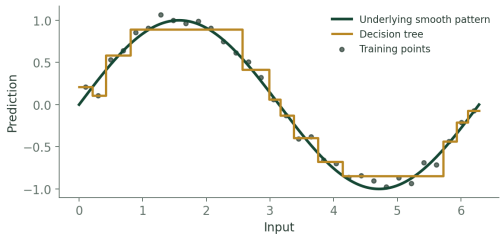
CS 4880

Kirk Duran

Where a Decision Tree Struggles

Trees partition the world into regions; some targets change smoothly across those regions.

- A standard decision tree predicts one value throughout each regression leaf: a **step-shaped approximation**.
- A curved or diagonal boundary may require many axis-aligned splits.
- Small changes in training records can produce a very different tree; ensembles reduce some of this instability.



Key idea

Neural networks learn connected numerical transformations rather than a sequence of hard feature tests.

Learning a Category Without a Rulebook

We can recognize many different dogs without writing a rule for every breed.

- A child sees large, small, fluffy, and short-haired dogs; examples establish a useful category despite differences.
- Correlated clues such as shape, ears, tails, and movement can collectively support recognition.
- In supervised learning, we give a model examples with known labels; it learns adjustable combinations of clues.
- The model may learn statistical associations rather than the real-world meaning of "dog."

Connect to learning agents

The agent's **learning element** updates a predictor using feedback. Its **performance element** can then use that predictor to select actions. A neural network is a model inside this larger loop, not an entire learning agent by itself.

Biological Inspiration: Neurons and Plasticity

The biological analogy is useful for intuition, but an artificial neuron is a mathematical simplification.

- Biological neurons receive signals through connections, integrate inputs, and can produce an output signal.
- The strength of connections can change through experience, a phenomenon called **synaptic plasticity**.
- The Hebbian slogan "neurons that fire together wire together" captures one form of association learning.

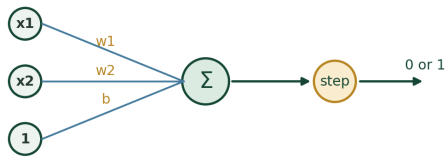
Artificial analogue

Inputs $\longrightarrow$ weighted sum $\longrightarrow$ activation $\longrightarrow$ output.

Training modifies numerical **weights** to improve predictions. Modern backpropagation is not intended as a literal model of brain learning.

The Perceptron: A Trainable Artificial Neuron

A perceptron makes a binary decision from a weighted vote of its inputs.



- Feature weights increase or decrease influence.
- Add the weighted inputs and a bias (the decision's starting offset).
- Threshold the score to obtain 0 or 1.

Weighted score followed by a threshold

$$z = b + \sum_{j=1}^{d} w_j x_j, \qquad \widehat{y} = \mathrm{step}(z).$$

d: feature count; j: feature index; x_j: input; w_j: weight; b: bias; z: weighted score; $\widehat{y}$: predicted label. Here $\mathrm{step}(z) = 1$ when $z \geq 0$, otherwise 0.

How a Perceptron Learns from Mistakes

When the label is wrong, adjust the weights toward the correct answer.

1. Compute the current prediction from the feature values.
2. Compare the predicted label with the actual training label.
3. If the prediction is correct, leave the weights unchanged for that example.
4. If it is wrong, change each weight in proportion to its input feature.

Per-example perceptron update

$$w_j \leftarrow w_j + \eta(y - \widehat{y})x_j, \qquad b \leftarrow b + \eta(y - \widehat{y}).$$

y: true 0/1 label; $\widehat{y}$: prediction; $\eta > 0$: learning rate; x_j: feature; w_j, b: weight and bias. $\leftarrow$: replace with the updated value. The error $y - \widehat{y}$ is $+1$ for a missed class-1 example, -1 for a false class-1 prediction, and 0 if correct.

Key idea

This is supervised learning: observed labels change the model rather than being stored as independent handwritten rules.

Prepare One Shared Train, Validation, and Test Split

Use the same 30 features and partitions for the perceptron and the later MLP.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X, y = load_breast_cancer(return_X_y=True)
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=.2, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev, test_size=.2, stratify=y_dev,
    random_state=42)
```

```
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

Key idea

Fit the scaler only on training rows. Dataset labels: $0 =$ malignant; $1 =$ benign. A sigmoid score for class 1 is a benign score, not a cancer-risk score.

Python Example: Breast-Cancer Perceptron

Fit a linear classifier, predict held-out labels, then measure accuracy.

```
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

clf = Perceptron(max_iter=1000,
                 tol=1e-3,
                 random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_val)
acc = accuracy_score(y_val, y_pred)
print(f"Validation Accuracy: {acc:.2f}")
```

What each call means

`fit`: learn the feature weights.

`predict`: apply the learned boundary to held-out records.

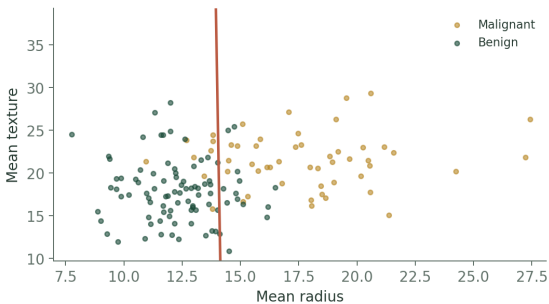
`accuracy_score`: count correct predictions.

Key idea

The previous slide supplies the scaled data. Compare this baseline on validation; keep the final test out of model and epoch selection.

What Can That Perceptron Represent?

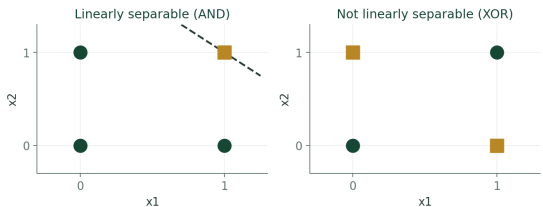
The learned weights tilt and move one linear boundary through the feature space.



- This supplied plot uses mean radius and mean texture; it is not the 30-feature model fitted by our code.
- A single boundary cannot wrap around disconnected or highly curved clusters.
- Better features can help; adding hidden layers changes what the model can represent.

XOR: A Simple Problem One Perceptron Cannot Solve

Some patterns have no single straight line separating the labels.



- In **XOR**, exactly one input must be 1.
- The positive cases are $(0, 1)$ and $(1, 0)$; the negative cases are $(0, 0)$ and $(1, 1)$.
- Every proposed straight line misclassifies at least one corner.

XOR truth table

$0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. $\oplus$ denotes exclusive OR; x_1, x_2 are binary inputs, w_1, w_2 are weights, and b is bias. No single affine threshold matches all four cases.

The First Wave: A Breakthrough and a Boundary

The perceptron made learning from data tangible, but its expressive limits were real.

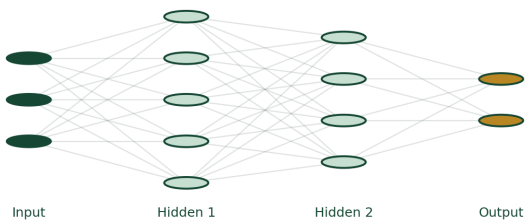
- **1958:** Frank Rosenblatt's perceptron popularized trainable pattern recognition, building on earlier threshold-neuron models.
- **1969:** Minsky and Papert analyzed important limits of single-layer perceptron architectures, including XOR-like problems.
- Enthusiasm for perceptrons faded; this contributed to broader skepticism surrounding early AI research.
- The first AI winter had *multiple* technical, funding, and expectation-related causes, not one mathematical counterexample.

Key idea

One layer was not expressive enough for every classification problem. The next question: can we combine simple units into a more capable model?

A Multilayer Perceptron Combines Learned Features

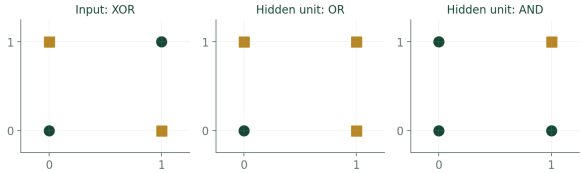
Hidden units can transform the original inputs before the final decision.



- **Input layer:** the feature values.
- **Hidden layers:** combinations of earlier signals, followed by nonlinear activations.
- **Output layer:** a class score, class probabilities, or a number.
- Stacking transformations can represent boundaries unavailable to one linear unit.

How Hidden Units Make XOR Possible

A hidden layer can ask simpler questions whose answers solve a harder one.



One hidden unit detects whether *either* input is on (OR); another detects whether *both* are on (AND). The output combines their answers: **XOR = OR and not AND**.

A small network implementing XOR with step activations

$$h_{\text{OR}} = \text{step}(x_1 + x_2 - 0.5), \quad h_{\text{AND}} = \text{step}(x_1 + x_2 - 1.5),$$
$$\widehat{y} = \text{step}(h_{\text{OR}} - h_{\text{AND}} - 0.5).$$

x_1, x_2: binary inputs; $h_{\text{OR}}, h_{\text{AND}}$: hidden outputs; $\widehat{y}$: prediction. Check $(1, 1)$: both hidden values are 1, so the output is 0. These weights are hand-chosen, not gradient-trained.

Inside One Hidden Unit

Every hidden unit repeats the same two steps: combine inputs, then activate.

1. Multiply each incoming feature by the weight on its connection.
2. Add those contributions and a bias.
3. Apply an activation function to the sum.
4. Send the resulting value to the next layer.

Forward computation for a hidden unit

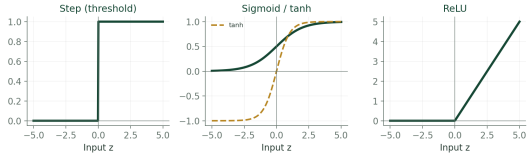
$$z_j = b_j + \sum_k w_{jk} a_k, \qquad a_j = \phi(z_j).$$

j: current unit; k: previous-layer unit; a_k: incoming activation; w_{jk}: connection weight; b_j: bias; z_j: weighted score; ϕ: nonlinearity; a_j: resulting activation. Sum over all incoming units.

Why nonlinear? Stacked affine transformations (linear weights plus biases) collapse to one affine transformation. Nonlinear activations change what can be represented.

Activations: Threshold, Sigmoid/Tanh, and ReLU

The activation determines how a unit responds to its combined input.



- **Step:** a hard 0/1 decision, useful for the original perceptron but unsuitable for ordinary gradient training.
- **Sigmoid/tanh:** smooth nonlinear functions; large positive/negative scores can cause nearly flat responses.
- **ReLU:** leaves positive inputs unchanged and sets negative inputs to zero; a practical default for many hidden layers.

Activation Formulas: What Each Curve Computes

The same weighted score can produce very different activations.

Common formulas

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}, \qquad \mathrm{ReLU}(z) = \max(0, z).$$

z: weighted input; e: Euler's number; σ: sigmoid; $\tanh$: hyperbolic tangent; ReLU: rectified linear unit; $\max$: take the larger value. ReLU has a kink at zero; implementations choose a derivative convention there.

Check one input: $z = -1$

Sigmoid gives about 0.269; tanh gives about -0.762; ReLU gives 0.

Key idea

Sigmoid is bounded between 0 and 1; tanh between -1 and 1; ReLU is unbounded above. A hidden activation is not automatically a class probability.

The Missing Piece: How Do Hidden Layers Learn?

The correct output tells us the network is wrong; it does not say which internal weight caused the error.

- A perceptron's output directly exposes its decision error.
- A multilayer network has many hidden weights affecting the same output.
- If the final prediction is wrong, should we change an early connection, a late connection, or both?
- Hard thresholds offer no useful local slope for gradient-based credit assignment.

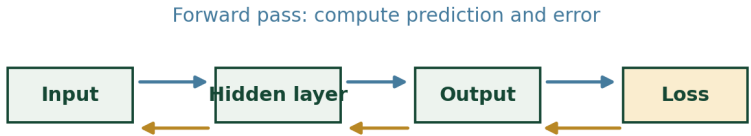
Historical distinction

Multilayer networks can **represent** XOR, even with step activations. But efficient, scalable **training** requires a way to assign responsibility for error across layers.

Representability is not trainability.

Backpropagation: Send Error Information Backward

Compute how a small change to each weight would affect the current loss.



- **Forward pass:** compute each activation and the final prediction.
- **Loss:** measure how different the prediction is from the label.
- **Backward pass:** compute the loss gradient with respect to each weight.
- **Optimizer:** use those sensitivities to update the weights.

The Chain Rule: Assigning Responsibility

A weight affects the loss through a chain of intermediate calculations.

- Picture a weight changing a hidden score, which changes an activation, which changes a prediction, which changes the loss.
- A **derivative** expresses how strongly one value changes as another changes slightly.
- Backpropagation multiplies the sensitivities along the chain so each weight gets a meaningful update direction.

For a weight w that affects a single downstream activation

$$\frac{\partial L}{\partial w} = \underbrace{\frac{\partial L}{\partial a}}_{\text{activation to loss score}} \quad \underbrace{\frac{\partial a}{\partial z}}_{\text{score to activation}} \quad \underbrace{\frac{\partial z}{\partial w}}_{\text{weight to score}} .$$

L: loss; w: one weight; z: pre-activation score; a: activation; $\partial L/\partial w$: local loss sensitivity to that weight, holding other weights fixed. If a value feeds several downstream paths, their gradient contributions are added.

Backpropagation and Gradient Descent Have Different Jobs

Backpropagation finds the sensitivities; the optimizer decides how to use them.

Backpropagation

Calculates how each weight influences the current loss using the chain rule.

Gradient descent / Adam

Changes weights in ways expected to lower loss; Adam adapts update sizes using gradient history.

One basic gradient-descent step

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial L}{\partial w}.$$

t: update index; $w^{(t)}, w^{(t+1)}$: old/new weight; $\eta > 0$: learning rate; $\partial L/\partial w$: gradient evaluated at the current weights. A batch is a group used for one update; an epoch is one pass through the training examples.

Key idea

Backpropagation is an efficient way to compute gradients for many connected parameters; it is not itself the weight-update policy.

1986 and the Return of Multilayer Learning

Efficient error propagation made multilayer networks trainable in practice.

- Work on reverse-mode differentiation and error propagation predates 1986.
- The influential 1986 work of **Rumelhart, Hinton, and Williams** helped popularize backpropagation for multilayer connectionist models.
- Smooth activations and computed gradients supplied a practical procedure for learning hidden representations.
- Better training algorithms did not immediately make deep networks dominant: computational power and useful datasets were still limited.

Key idea

The conceptual breakthrough was an algorithm to **train** internal layers, not merely a drawing of a deeper network.

Why Deep Networks Still Struggled in the 1990s

A successful training principle still depends on data, compute, and numerical behavior.

- **Data:** many tasks lacked large, labeled datasets.
- **Compute:** training many layers with the available hardware was expensive.
- **Vanishing gradients:** saturated sigmoid/tanh units could pass tiny update signals through many layers.
- **Optimization and initialization:** training could proceed slowly or unpredictably.

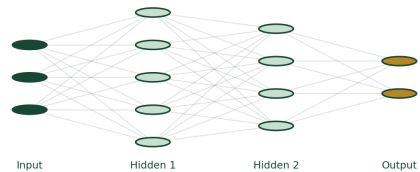
Historical context

The late-1980s/early-1990s AI funding downturn had several causes, including disappointment with expert systems. Neural-network research continued; these periods should not be explained by a single architecture or training problem.

The 2010s: Data, Compute, and Better Training

Deep learning became practically effective when several enabling conditions came together.

- **Larger datasets:** ImageNet, speech corpora, and other large collections supported richer models.
- **Accelerators:** GPUs and distributed computation made larger training runs possible.
- **Improved training:** ReLU, better initialization, normalization, regularization, and more capable optimizers helped stabilize learning.

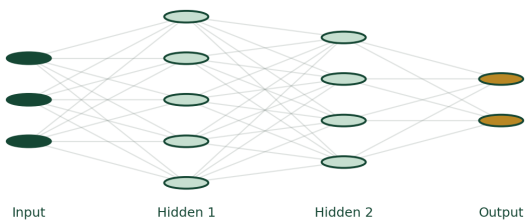


Key idea

Deep learning was not one new neuron formula: it was old core ideas plus advances in data, hardware, architecture, and optimization.

A Modern Feedforward Network: What Gets Learned?

Training adjusts weights and biases; the architecture specifies how they are connected.



- **Parameters:** weights and biases fitted from labeled examples.
- **Hyperparameters:** number of layers, units, activation functions, batch size, epochs, learning rate.
- **Loss:** tells us how far the outputs are from the desired targets.
- **Validation:** helps assess whether improvements extend to unseen examples.

Breast-Cancer Example: Reuse the Same Evaluation Protocol

Change the model, keeping the data and evaluation rules fixed.

- Use the training, validation, and test arrays prepared before the perceptron example.
- Every record is a vector of 30 numerical measurements.
- The scaler was fitted only on training rows; validation and test use those same training statistics.
- The MLP learns from training rows and monitors validation rows; neither model trains on the final test.

Key idea

This lets us compare model families fairly. The data label 1 still means benign; increasing a sigmoid score does not mean increasing malignancy risk.

Breast-Cancer Example: Define the Neural Network

The architecture stacks two nonlinear hidden layers and one binary output.

```
import tensorflow as tf
tf.keras.utils.set_random_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(30,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

- `Input(30)`: 30 measurements.
- `Dense(64)`: 64 learned feature combinations.
- `Dense(32)`: 32 more transformations.
- `Dense(1)` with sigmoid: one score between 0 and 1.

Breast-Cancer Example: Compile the Model

Compilation specifies what error means and how the weights will change.

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

- **Loss:** binary cross-entropy penalizes confident probability mistakes.
- **Optimizer:** Adam uses backpropagated gradients to update trainable parameters.
- **Metric:** accuracy counts correct labels at the evaluation threshold.

Loss for one binary-labeled observation

$$\ell(p, y) = -\Big[y \ln p + (1 - y) \ln(1 - p)\Big].$$

$y \in \{0, 1\}$: true label; $p \in (0, 1)$: predicted probability of class 1; ℓ: one-example loss; $\ln$: natural logarithm. For $y = 1$, loss is $-\ln p$; for $y = 0$, it is $-\ln(1 - p)$. L usually averages such losses over a batch.

Breast-Cancer Example: Fit for 30 Epochs

An epoch is one pass through the training examples; batches make several updates per epoch.

```
history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=16,
    validation_data=(X_val, y_val),
    verbose=1
)
```

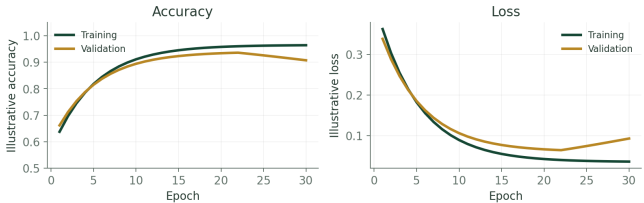
- `batch_size=16`: update weights after processing groups of sixteen examples.
- `validation_data`: monitor the separate validation rows without training on them.
- `history`: stores epoch-by-epoch loss and accuracy for plotting.

Key idea

Freeze settings chosen using validation, then report `model.evaluate(X_test, y_test)` once.

Reading Training Histories

Look for generalization, not only rising training accuracy.



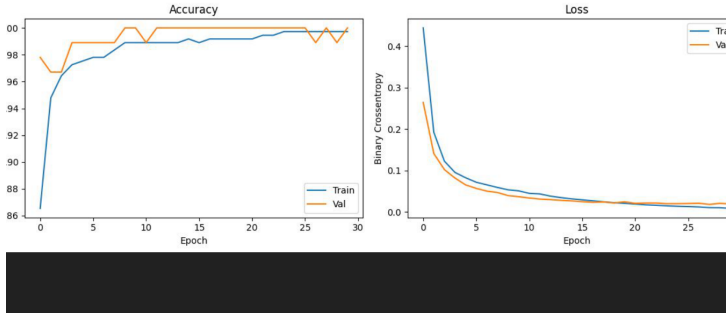
- Accuracy should improve as useful relationships are learned, but fluctuations are normal.
- If training loss falls while validation loss rises, the model may be overfitting.
- A high validation accuracy in a small sample is not a guarantee of future reliability.

Retrieving the arrays returned by Keras

`history.history['loss']` and `history.history['val_loss']` are loss curves; the `accuracy` and `val_accuracy` entries are classification curves.

Compare the Supplied Training History Carefully

A supplied result illustrates a run; it is not a guaranteed outcome of our revised split.



- Both validation curves improve here; compare this with the illustrative overfitting curves on the previous slide.
- Split choices, preprocessing, random seeds, and training details affect the numbers.

Continuous Outputs and Universal Approximation

A neural network is not limited to classification: it can predict continuous quantities.

- Replacing the final sigmoid with a linear output lets a network predict a numerical value.
- With a suitable nonlinear activation and enough hidden units, certain neural networks can approximate any continuous target on a bounded input region.
- This is an **existence result**, not a promise that a particular dataset or optimizer will find the desired approximation.
- Depth can provide more efficient representations for some structured functions; it is not automatically superior on every task.

The theorem's practical reading

For a continuous target g on a closed, bounded region K, suitable networks can be constructed such that

$$\max_{x \in K} |f_\theta(x) - g(x)| < \varepsilon \qquad \text{for any chosen } \varepsilon > 0.$$

K: compact (closed, bounded) input set; x: input; g: continuous target; f_θ: network output; θ: all weights and biases; $\varepsilon > 0$: chosen error tolerance. $\max$ measures the worst absolute error throughout K. The result does not guarantee a feasible size or a successful training procedure.

Original Python Example: Learning a Sine Wave

Use the same feedforward architecture for a numerical target instead of a class label.

```
import numpy as np
import tensorflow as tf

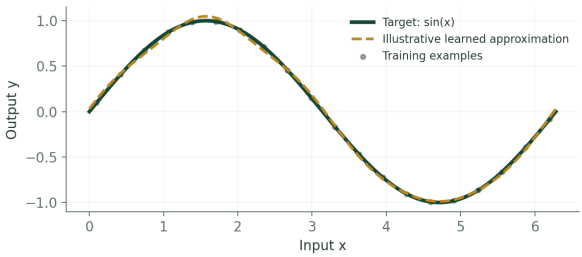
X = np.linspace(0, 2 * np.pi, 1000).reshape(-1, 1)
y = np.sin(X)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(64, activation='tanh'),
    tf.keras.layers.Dense(64, activation='tanh'),
    tf.keras.layers.Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=100, batch_size=32, verbose=0)
```

Key idea

The last layer has **no sigmoid**, so it can produce any real number. Mean squared error measures numerical prediction differences.

A Network Can Fit a Smooth Function

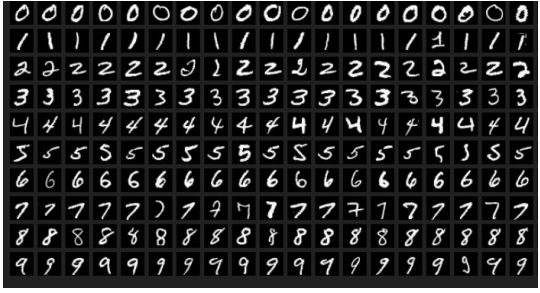
The network learns to combine smooth nonlinear transformations into a smooth prediction.



- The input is one position x; the target is $\sin(x)$. The supplied curve is illustrative, not a rerun.
- Two hidden layers with `tanh` allow a curved numerical relationship.
- Training MSE asks how well the approximation matches observed targets.
- Evaluation on new inputs is needed to assess interpolation and generalization.

MNIST: Recognizing Handwritten Digits

The same model family can classify images by treating their pixel values as features.



- Each grayscale image has $28 \times 28 = 784$ pixels.
- Flatten the image to a vector of 784 input values.
- Normalize intensities from 0–255 into the 0–1 range.
- Predict one of ten classes: the digits 0 through 9.

Original Python Example: A Dense MNIST Classifier

Replace the binary sigmoid with a ten-way softmax output.

```
(X_train, y_train), (X_test, y_test) = (
    tf.keras.datasets.mnist.load_data())
X_train = X_train.reshape(-1, 784).astype('float32') / 255.0
X_test  = X_test.reshape(-1, 784).astype('float32') / 255.0
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

Why softmax at the output?

$$p_k = \frac{e^{z_k}}{\sum_{j=0}^{9} e^{z_j}}, \qquad \sum_{k=0}^{9} p_k = 1.$$

z_k: raw output score (logit) for digit k; e: Euler's number; $j, k \in \{0, \ldots, 9\}$: class indices; p_k: estimated class probability. Choose the largest p_k; normalization alone does not guarantee calibration.

MNIST: Train With Integer Labels and Test Once

Keep a validation subset of the 60,000 training images; preserve the official test set.

```
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=10000,
    stratify=y_train, random_state=42)
model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=3, batch_size=128,
    validation_data=(X_val, y_val), verbose=0)
print("MNIST test:", model.evaluate(X_test, y_test, verbose=0))
```

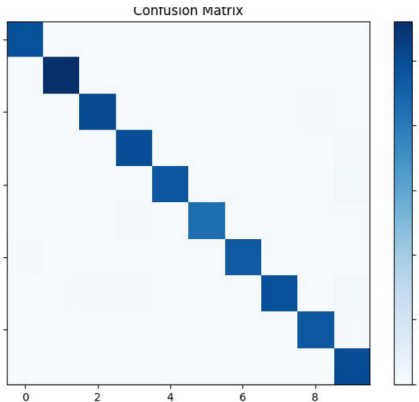
- `sparse_categorical_crossentropy` expects integer labels 0 through 9, not one-hot vectors.
- Three epochs is a preset demonstration budget; use validation if you change it.

Key idea

The supplied confusion matrix is a separate earlier result. A newly trained model must be evaluated on its own predictions.

MNIST: Evaluate Confusion, Not Only Accuracy

A high overall score can still hide systematic confusion between particular digits.

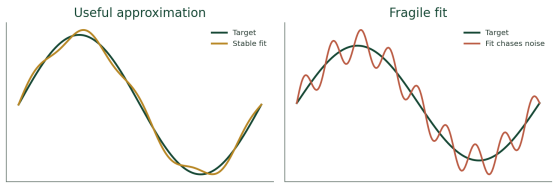


- **Diagonal cells:** images assigned the correct digit.
- **Off-diagonal cells:** confusion between two digits.
- This supplied chart is not a result of our new run. Axis direction and counts must be labeled before reporting which digits are confused.
- Flattening retains pixel positions as vector indices. Dense layers do not build in local neighborhoods or shared filters; CNNs do.

Downsides of Neural Networks

More expressive models come with costs and failure modes.

- Large labeled datasets may be needed to fit many parameters reliably.
- Deep internal representations are harder to interpret than short decision rules.
- Larger networks may demand substantial computation and memory.
- Overfitting can still occur, especially with small or noisy samples.
- Prediction confidence is not automatically calibrated uncertainty.
- Training depends on architecture, initialization, learning rate, and other choices.



Hyperparameters: What We Choose Before Training

Learn weights from data; use validation evidence to choose the training design.

Choice	What it controls	Common examples
Layers, units	Network depth and width	2 layers; 64 or 128 units
Activation	How a neuron transforms input	ReLU, tanh, sigmoid
Learning rate / optimizer	How fast and how weights change	Adam, SGD
Batch size / epochs	Updates per pass; passes through data	16, 32 / 30, 100
Regularization	Controls overly complex fits	Dropout, weight decay
Stopping	When training should end	Early stopping

Key idea

Use training data to **fit** weights, validation data to **choose** hyperparameters, and a final untouched test set to **evaluate** the selected model.

From One Perceptron to Deep Learning

1. A perceptron learns a linear boundary by changing feature weights after mistakes.
2. Hidden layers make more complex functions possible, including XOR.
3. Nonlinear activation functions provide useful intermediate representations and gradients.
4. Backpropagation calculates weight sensitivities; the optimizer updates the parameters.
5. Data, accelerators, and improved training methods made deep learning practical at larger scale.
6. The same basic framework supports binary classification, continuous regression, and multiclass image recognition.

Key idea

Representability asks *what the network could express*; optimization asks *what training can find*; generalization asks *whether it works on new cases*.

Next: architectures that exploit structure in images and sequences (CNNs and RNNs); generative networks are another extension.